

Full length article

A rollout heuristic-reinforcement learning hybrid algorithm for disassembly sequence planning with uncertain depreciation condition and diversified recovering strategies

Yaping Ren^{a,b,c,*}, Zhehao Xu^c, Yanzi Zhang^d, Jiayi Liu^e, Leilei Meng^f, Wenwen Lin^g

^a Beijing Institute of Technology, Zhuhai, China

^b Greater Bay Area Innovation Research Institute of BIT, Zhuhai, China

^c School of Intelligent Systems Science and Engineering, Jinan University, Zhuhai, China

^d School of Mechanical Engineering, Beijing Institute of Technology, Beijing, China

^e School of Information Engineering, Wuhan University of Technology, Wuhan, China

^f School of Computer Science, Liaocheng University, Liaocheng, China

^g Faculty of Mechanical Engineering and Mechanics, Ningbo University, Ningbo, Zhejiang, China

ARTICLE INFO

Keywords:

Disassembly sequence planning
Uncertain depreciation condition
Diversified recovering strategies
Rollout heuristic
Reinforcement learning

ABSTRACT

Disassembly is one of the crucial aspects of green manufacturing. For the end-of-life products, an effective disassembly sequence planning method can enhance recovery value and mitigate the negative consequences of resource depletion and waste generation. However, both the uncertainty of product depreciation condition and the NP-hard characteristics (including the determination of disassembly sequences and the selection of recovering strategies of subassemblies) of the disassembly sequence planning results in difficulties to determine the optimal/near-optimal disassembly solutions. To address these challenges, this work establishes an extended Petri net that considers diversified recovering strategies of each subassembly caused by uncertain product depreciation condition. Then, a rollout heuristic-reinforcement learning hybrid algorithm that integrates a rollout decision rule into the reinforcement learning procedure is proposed to rapidly find the high-quality disassembly solutions based on the extended Petri net, in which the uncertainty of disassembly information is tackled by training disassembly samples and the global exploration capability of the learning procedure is significantly improved by using the rollout decision rule. Finally, three products with different complexities and sizes are used to verify the performance of the proposed algorithm, and the experimental results indicate that our proposed rollout heuristic-reinforcement learning hybrid algorithm can efficiently compute the high-quality disassembly sequences under various disassembly environments.

1. Introduction

Driven by Industry 4.0, the electronics manufacturing industry is undergoing significant transformation, with the application of various advanced technologies making production processes more intelligent and efficient [1,2]. However, the excess production capacity has significantly shortened the product lifecycles, resulting in acceleration of obsolescence [3]. The United Nations highlighted in the “Global E-waste Monitor 2020” report that the total global e-waste reached 53.6 million metric tons, but only 17.4 % of electronic waste was adequately processed [4]. Some legislative methods have been proposed to address the issue such as waste electrical and electronic equipment (WEEE) [5]

and extended producer responsibility (EPR) [6]. Nevertheless, most of the recyclable materials contained in the end-of-life (EOL) products cannot be effectively reused and pile up in landfills, leading to resource wastage [7,8]. The heavy metals without recovery also pose a significant threat to the environment and human health [9]. Therefore, taking proactive actions to achieve green manufacturing is urgent.

Disassembly is the process of removing or separating valuable or hazardous subassemblies including parts and components from EOL products [10], which forms disassembly sequences based on various constraints, e.g., the priority relationships between subassemblies. The optimal disassembly sequences should be determined to mitigate environmental pollution and conserve energy, with the maximization of

* Corresponding author.

E-mail address: renyp1@163.com (Y. Ren).

<https://doi.org/10.1016/j.aei.2024.103082>

Received 29 May 2024; Received in revised form 15 October 2024; Accepted 22 December 2024

Available online 3 January 2025

1474-0346/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

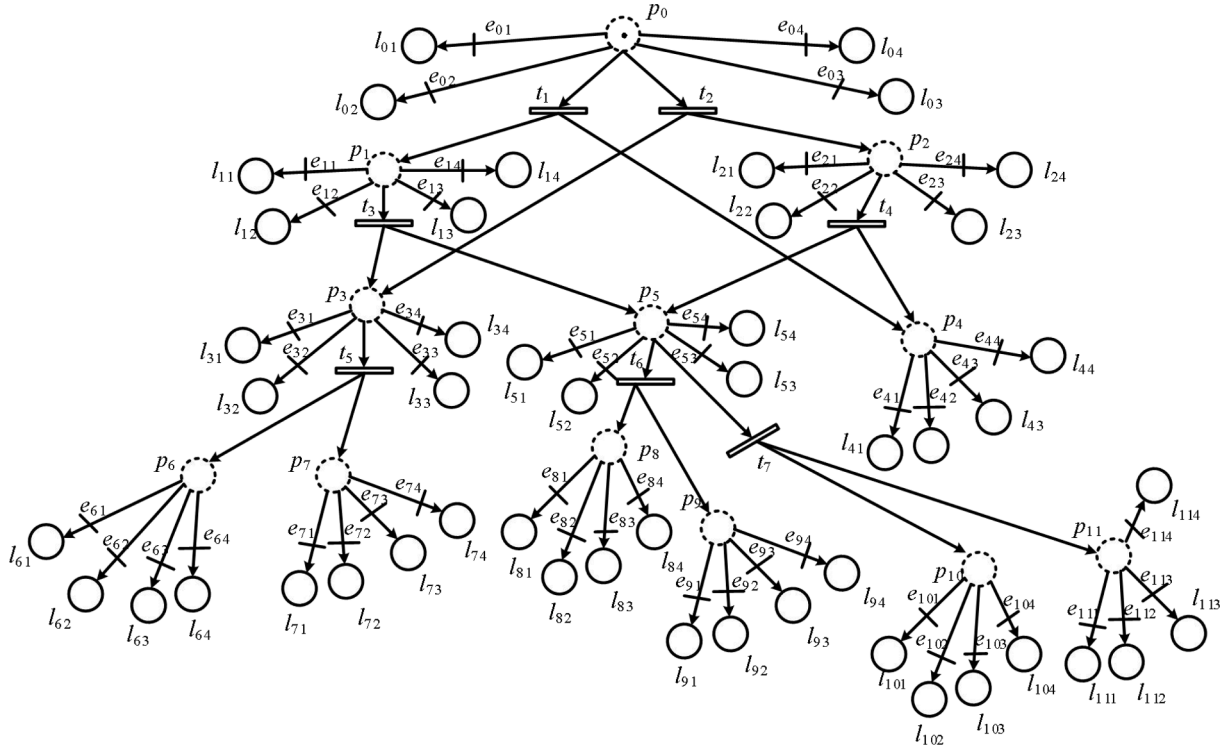


Fig. 1. The EPN of a product.

recovering value. This decision-making process is termed as disassembly sequence planning (DSP) problem [11,12], which involves two major issues: product modeling and disassembly sequencing [13].

A product to be disassembled must be represented and modeled based on its assembly structure, subassembly connectivity, physical constraints and other disassembly information before the DSP. Graph-based modeling methods, e.g., precedence graph [14], AND/OR graph [15–18], and Petri net [19–21], are widely employed in the representation of products. The selection of product modeling methods should take disassembly requirements into consideration, which directly affects the subsequent disassembly sequencing. Petri net is a kind of efficient graphical modeling method for discrete event systems, which can be applied to denote disassembly sequence information. The current research expands Petri net to incorporate more comprehensive disassembly information. For instance, Guo et al. considered disassembly time when modeling the Petri net [22]. Mao et al. introduced an extended stochastic Petri net, incorporating probability distributions to address the randomness of disassembly time and cost [21].

During the disassembly processes of EOL products, the efficacy of predetermined disassembly solutions is compromised by the inherent uncertainty of product depreciation condition, which can be resulted from different component/part failures, such as wear, fracture, deformation, and corrosion [23]. The selection of recovering strategies (including reuse, remanufacturing, recycling, and disposal) of subassemblies (i.e., components and parts) in an EOL product is determined by the product depreciation condition. However, there are few DSP studies that consider the influence of the uncertainty of product depreciation condition on the diversification of recovering strategies of subassemblies. Therefore, this paper studies a DSP with Uncertain depreciation condition and Diversified recovering strategies (DSP-UD), in which the uncertainty of product depreciation condition and the diversification of recovering strategies are simultaneously taken into account. In addition, due to the extendibility of Petri net for the disassembly modeling, an Extended Petri Net (EPN) is established to depict and formulate the disassembly information, including the relations between subassemblies and disassembly operations, the uncertain product

depreciation condition, and the diversified recovering strategies.

Apart from the uncertainty of product depreciation condition, a computational challenge for solving DSP-UD exists due to its NP-hard characteristics, in which not only the determination of disassembly sequences but also the selection of recovering strategies of subassemblies should be addressed [24]. This means that the computational complexity of DSP-UD exponentially increases with the number of subassemblies and disassembly operations. Many (meta)heuristic methods have been proposed to solve uncertain DSPs which are able to effectively search for the near-optimal disassembly sequences within an acceptable computational time. However, they often require some prior knowledge to address the disassembly uncertainty [25]. For example, Tian et al. analyzed the uncertainty of disassembly cost and time by fuzzy programming and used the artificial bee colony algorithm to determine the optimal disassembly sequence [26]. Similarly, Gao et al. apply the artificial bee colony algorithm to address the uncertainty of components and disassembly transitions based on stochastic and fuzzy assessment data [27]. Zhang et al. adopts trapezoidal fuzzy numbers to represent uncertain information and propose a hybrid approach termed nondominated sorting genetic algorithm-II with simulated large neighborhood search to solve the selective DSP problem [28]. It can be seen from these uncertain DSP studies that the uncertain disassembly factors or parameters are generally represented by some predetermined probability distributions or fuzzy values.

Recently, reinforcement learning (RL) methods have attracted extensive attention for solving uncertain DSPs [29], which not only effectively deal with the uncertainty of DSP with little prior disassembly knowledge but also efficiently identify the high-quality disassembly solutions. Furthermore, different from the (meta)heuristic methods that randomly decide on every disassembly operation according to the physical constraints (e.g., precedence relationships), every disassembly operation is selected out based on not only the physical constraints but also the expected reward-to-go from the current stage in RL, which is prone to find better disassembly sequences. Zhao et al. employed deep Q-Network to address the DSP considering high structural uncertainty, resulting in a non-dominated solution set and a high-quality Pareto front

[30]. Mao et al. introduce the deep Q-network to address the problem of unpredictable disassembly targets and apply in the pre-employment training [31]. Amirnia et al. developed a multi-agent deep Q-network for disassembly planning to adapt to random unexpected events during the disassembly process and improve disassembly performance under uncertain conditions [32]. However, these existing RL methods are limited to the action selection strategies during the training/learning process [33,34]. Specifically, they often adopt a single probability-based selection strategy, such as the ϵ -greedy strategy and Boltzmann strategy [30,32], and the future reward-to-go is estimated at every decision stage without any lookahead step. In contrast, our proposed RRHA integrates a rollout heuristic into the action selection of RL, in which the rollout heuristic is used to estimate the future reward-to-go of alternative actions by looking ahead from the current stage to certain future stages, thereby enhancing the estimation accuracy of the future reward-to-go and the quality of the selected action [35,36]. To fill the gap, a Rollout heuristic-Reinforcement learning Hybrid Algorithm (RRHA) algorithm that integrates a rollout decision rule into the RL framework is proposed to efficiently address DSP-UD, where the uncertain depreciation condition and recovering strategies are effectively tackled by training/learning disassembly samples. The proposed hybrid algorithm combines the advantages of reinforcement learning and rollout heuristics, where the exploration ability of learning procedure is significantly enhanced by using the rollout decision rule.

The main contributions of this paper are given as follows:

- 1) An eight-tuple EPN is proposed to model DSP-UD, in which the uncertain depreciation condition and diversified recovering strategies are simultaneously taken into account.
- 2) A hybrid algorithm that integrates RL with a rollout heuristic, i.e., RRHA is developed to efficiently solve DSP-UD, where a Markov Decision Process (MDP) for DSP-UD is modeled based on the EPN.
- 3) Finally, two existing RL algorithms and three existing (meta)heuristic approaches are applied to compare with RRHA, and the comparison results indicate that RRHA is significantly easier to identify the best disassembly solution than these existing methods.

The rest of this paper is organized as follows. In [section 2](#), the EPN is established in detail. [Section 3](#) illustrates RRHA. [Section 4](#) shows the experimental results. Finally, section 5 concludes this work.

2. Proposed EPN model

Petri net is a graphic modeling method and commonly applied to the representation of discrete event systems. For an introduction to basic Petri net, a reader can refer to [37]. The recovering strategy adopted for EOL products will change according to situation of component/part failures. To accommodate more uncertain disassembly and recycling information in EOL products, we consider the diversified recovering strategies based on product depreciation to formulate an EPN model with following definition based on the basic Petri net by expanding the EOL options after disassembly.

The example of EPN is shown in [Fig. 1](#). Each circle represents a place. The dashed circles are original places, representing subassemblies or parts that may appear during the disassembly process. Unlike the simple Petri nets, the EPN model uses solid circles as leaf places derived from each original place to represent four alternative EOL options. These EOL options determine how the corresponding subassemblies or parts will be processed by the bar arrows connecting the original place and the leaf places. The directed arcs represent transitions, indicating the operation of disassembly or the selection of EOL options. The EPN model can be represented as an eight tuple:

$$\text{EPN} = (P = P_O \cup P_L, T = T_D \cup T_E, F = I \cup O, s, \Psi, \delta, \xi, \tau),$$

where

Table 1

Input matrix I_m .

a_{ij}	1	2	3	4	5	6	7
0	1	1	0	0	0	0	0
1	0	0	1	0	0	0	0
2	0	0	0	1	0	0	0
3	0	0	0	0	1	0	0
4	0	0	0	0	0	1	0
5	0	0	0	0	0	0	1
6	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0

Table 2

Output matrix O_m .

b_{ji}	0	1	2	3	4	5	6	7	8	9	10	11
1	0	1	0	0	1	0	0	0	0	0	0	0
2	0	0	1	1	0	0	0	0	0	0	0	0
3	0	0	0	1	0	1	0	0	0	0	0	0
4	0	0	0	0	1	1	0	0	0	0	0	0
5	0	0	0	0	0	0	1	1	0	0	0	0
6	0	0	0	0	0	0	0	0	1	1	0	0
7	0	0	0	0	0	0	0	0	0	0	1	1

- 1) $P = P_O \cup P_L$ is the set of places and is separated into two subsets i.e. $P_O = \{p_0, \dots, p_i, \dots, p_n\}$ and $P_L = \{l_0, \dots, l_i, \dots, l_n\}$. Place p_i is an original place resulting from disassembly operations on the subassembly or part i , at which its EOL option is not determined. Place $l_i = \{l_{i1}, l_{i2}, l_{i3}, l_{i4}\}$ is the set of leaf places (i.e., terminal places) derived from p_i , representing four alternative EOL options depending on product depreciation. l_{ik} indicates the original place p_i is processed with EOL option k , $k = 1$ (disposal), 2 (recycling), 3 (remanufacturing), and 4 (reuse).
- 2) $T = T_D \cup T_E$ is the set of transitions and is partitioned to two subsets i.e. $T_D = \{t_1, \dots, t_j, \dots, t_m\}$ and $T_E = \{e_0, \dots, e_i, \dots, e_n\}$. T_D indicates the disassembly relationship between original places. Transition t_j represents an independent disassembly operation that separates an original place into more than one original place. T_E indicates the disassembly relationship from the original places to the EOL options. Similar to place $l_i \in P_L$, transition $e_i = \{e_{i1}, e_{i2}, e_{i3}, e_{i4}\}$ also consist of four alternative EOL operations, corresponding to l_i .
- 3) $F = I \cup O$ is a function that consists of inflow (input from places to transitions) and outflow (output from transitions to places) relations in the EPN. Inflow function $I(p_i, t_j)$ depicts the existence of the arc from any place $p_i \in P_O$ to transition $t_j \in T_D$. $I(p_i, t_j) = 1$ if t_j originates from p_i ; otherwise $I(p_i, t_j) = 0$. Similarly, outflow function $O(t_j, p_i)$ depicts the existence of the arc from any transition $t_j \in T_D$ to place $p_i \in P_O$. $O(t_j, p_i) = 1$ if t_j directs to p_i ; otherwise $O(t_j, p_i) = 0$. Finally, the inflow and outflow function can be formulated as matrix $I_{n \times m} = \{a_{ij}\}$ and $O_{m \times n} = \{b_{ji}\}$, respectively. a_{ij} and b_{ji} are defined as:

$$a_{ij} = \begin{cases} 1, & \text{if transition } t_j \text{ originates from place } p_i \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

$$b_{ji} = \begin{cases} 1, & \text{if transition } t_j \text{ directs to place } p_i \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

For example, the $I_{n \times m}$ and $O_{m \times n}$ corresponding to [Fig. 1](#) can be represented in the form of the following matrix [Table 1](#) and [Table 2](#).

- 4) $s = \{s_0, \dots, s_i, \dots, s_n\}$ is a vector that represents the disassembly state, where $s_i \in \{0, 1\}$ corresponds to the state of original place p_i . The original places other than p_0 will be assigned a token after the execution of its upstream transition, and the token will be removed

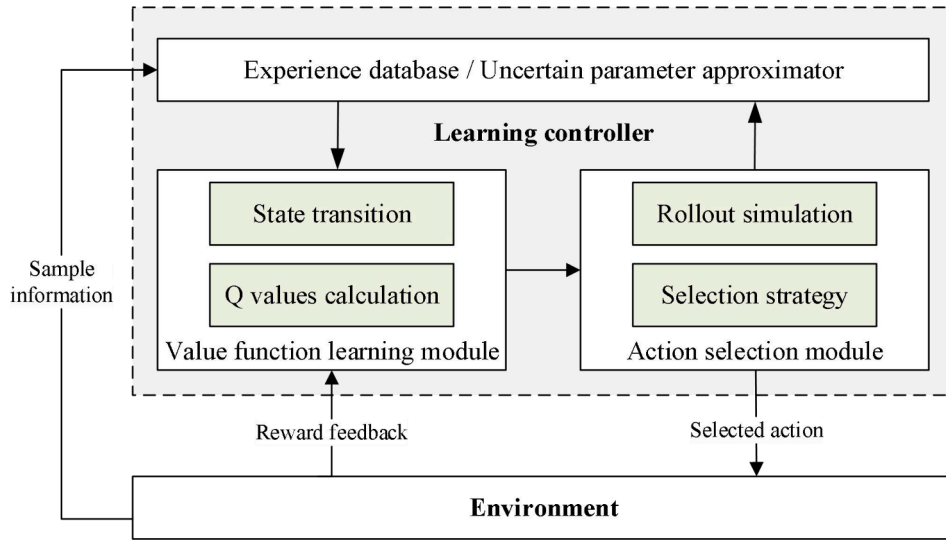


Fig. 2. The architecture of RRHA.

after the execution of one of its downstream transitions. In addition, a token is assigned to p_0 at the initial state. If the original place p_i holds the token, s_i equals 1; otherwise s_i equals 0. Notice that only the disassembly process is considered here, i.e., the terminal places in P_L are not taken into account. For example, the initial state s_0 would be formulated as $\{1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0\}$ in the situation that original place p_0 holds the token (denoted by a dot in Fig. 1). After the execution of transition t_2 , the token in p_0 will be removed and allocated to p_2 and p_3 , i.e., formulated as $\{0, 0, 1, 1, 0, 0, 0, 0, 0, 0\}$. The current state s maps to the next state s' through a selected transition t_j . This conversion could be formulated as a state transition function shown below, where $[j]$ represents the j^{th} row of the matrix, corresponding to the transition t_j :

$$s' = s + (O_{m \times n}[j] - I_{n \times m}^T[j]) \quad (3)$$

5) The product depreciation condition for each original place $p_i \in P_O$ is uncertain during disassembly. Function ψ is formulated to represent the depreciation of each original place p_i . $\psi(p_i)$ indicates the depreciation coefficient of place p_i , with the value from 0 (i.e., severe depreciation) to 1 (i.e., no depreciation). Meanwhile, the depreciation coefficients of original places are equally divided into four levels and correspondingly mapped to the selection of four EOL options according to Equation (4), where (p_i) indicates that place p_i is processed with the k^{th} EOL option for recovering. Noting that symbol “ $\lceil \cdot \rceil$ ” serves the function of ceiling. For example, the original places with depreciation coefficient range of 0 to 0.25 are classified as the worst level, so they are mapped to the 1st EOL option (i.e., disposal) through ceiling.

$$k(p_i) = \lceil 4\psi(p_i) \rceil \quad (4)$$

6) $\delta: P_O \times T_E \rightarrow [0, 1]$ depicts the discrete probability distribution of choosing different EOL options. For any original place $p_i \in P_O$, there are four alternative EOL operations $e_i = \{e_{i1}, e_{i2}, e_{i3}, e_{i4}\}$, where $\delta(p_i, e_{ik})$ indicates the probability that original place p_i is processed by the EOL operation e_{ik} .

7) ξ formulates the revenue function that recovering from the places. $\xi(p_i)$ depicts the expected revenue obtained by selling the original place p_i . The various EOL options result in different recovering revenue, which are fixed values derived from second hand market or remanufacturers. For any place p_i , $\xi(l_{ik})$ depicts the revenue obtained by place p_i executing the EOL option k , where the probability

associated with the EOL option k is weighted in combination to calculate the expected revenue shown in Equation (5).

$$\xi(p_i) = \sum_{k=1}^4 \delta(p_i, e_{ik}) \bullet \xi(l_{ik}), \forall i \quad (5)$$

8) τ formulates the cost function that consuming for transition. For any transition $t_j \in T_D$, $\tau(t_j)$ indicates the fixed cost of the corresponding disassembly process. For any transition $e_i \in T_E$, $\tau(e_i)$ indicates the expected cost of EOL operations that p_i are optional for the original place p_i . $\tau(e_{ik})$ indicates the cost of k^{th} EOL operations that are optional for the original place p_i . The calculation of expected $\tau(e_i)$ is shown in Equation (6).

$$\tau(e_i) = \sum_{k=1}^4 \delta(p_i, e_{ik}) \bullet \tau(e_{ik}), \forall i \quad (6)$$

A disassembly process starts from the original place p_0 and continuously selects the next feasible transition based on the EPN constraints to enter the next disassembly state. This process can be terminated at any time step or when there are no feasible transitions available in the current state, which compose a sequence of transitions, resulting in the final state. The objective of the DSP-UD addressed in this study is to find a best disassembly sequence of transitions that maximizes the recovering profit. Therefore, an evaluation function Val is formulated for the calculation of recovering profit throughout the entire disassembly process in Equation (7) to measure the disassembly performance:

$$Val = \sum_{i=1}^n [\xi(p_i) - \tau(e_i)] \bullet s_i - \sum_{j=1}^m \tau(t_j) \quad (7)$$

Taking the EPN model in Fig. 1 as an example and executing the transitions $t_1 \rightarrow t_3 \rightarrow t_5$ in sequence, the final state is terminated at $\{0, 0, 0, 0, 1, 1, 1, 1, 0, 0\}$, which includes the original places $\{p_4, p_5, p_6, p_7\}$. The function Val will calculate the profit yielded from $\{p_4, p_5, p_6, p_7\}$, where considering the expected cost from the transitions of EOL option and disassembly operation, i.e., the cost of $\{e_4, e_5, e_6, e_7\}$ and $\{t_1, t_3, t_5\}$.

3. The proposed RRHA

RL is a branch of machine learning that driving an agent to interact over time with the environment and learns how to take actions (i.e., the transitions in EPN) to maximize cumulative rewards through trial and error. In RL, the agent selects actions based on the current state and

evaluates the quality of actions based on the rewards or penalties received from the environment. By continuously interacting with the environment and learning from the outcomes, the agent improves its strategy to achieve better performance.

In this study, we develop a RRHA based on RL and rollout decision rule to address the EPN model which considering the diversified recycling strategy proposed in the previous section. The environment, state, and action of the RRHA correspond to the EPN disassembly model, disassembly state, and disassembly operation, respectively. The basic architecture of RRHA, as illustrated in Fig. 2, incorporates rollout decision rule into the framework of reinforcement learning. Similar to Q -learning algorithm, RRHA is also a model-free RL method that maximizes expected reward only through real experience, i.e., historical observations and feedback. On one hand, the uncertain parameter, i.e., $\delta(p_i, e_{ik})$ is approximated according to the previous sample information. On the other hand, the RRHA improves the quality of action selection when collecting the historical information of disassembly samples. Specifically, the RRHA applies the rollout heuristic at each decision stage to estimate the future expected reward-to-go of alternative actions by looking ahead from the current state to certain future states, thereby achieving more precise action selection.

Subsequently, we describe the details of the proposed RRHA based on the EPN in the following Section 3.1 to Section 3.5, including the uncertain parameter approximator, learning method of Q value function, selective sampling rule of disassembly sequences, selection method of disassembly solution and implementation procedure of RRHA.

3.1. Uncertain parameter approximator

The depreciation condition and recovering strategies of EOL products are difficult to predict in advance. Therefore, we introduce an uncertain parameter approximator that utilizes historical sample information to determine the frequency distribution of selecting different EOL options as probability distribution, i.e., $\delta(p_i, e_{ik})$ based on maximum likelihood estimation (MLE). MLE is a statistical method that calculates the parameter value that maximizes the probability of the occurrence of currently observed data. The parameter $\delta(p_i, e_{ik})$ is approximated once by MLE after each iteration, and its approximate value is applied to the reward computation in the next iteration.

Let W be the number of currently collected samples and $\Omega = \{\Omega_1, \dots, \Omega_w, \dots, \Omega_W\}$ be the experience database. Ω_w represents information of the w^{th} sampled disassembly sequence. Two auxiliary variables ω_{ik}^w and ϖ_i^w are defined p_i in Equation (8) and Equation (9), respectively. They are binary variables that ω_{ik}^w indicates whether the k^{th} EOL operation is selected to process the original place p_i and ϖ_i^w indicates whether the original place p_i holds a token in the sample Ω_w . Consequently, the approximate value of $\delta(p_i, e_{ik})$ can be formulated as Equation (10).

$$\omega_{ik}^w = \begin{cases} 1, & \text{if EOL operation } e_{ik} \text{ is selected in } \Omega_w \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

$$\varpi_i^w = \begin{cases} 1, & \text{if place } p_i \text{ holds a token in } \Omega_w \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

$$\delta(p_i, e_{ik}) = \frac{\sum_{w=1}^W \omega_{ik}^w}{\sum_{w=1}^W \varpi_i^w}, \quad \forall i \text{ and } k \quad (10)$$

3.2. The learning method of Q value function

The value function serves as a critical component for evaluating and guiding the action selection by converging Q values. In this subsection, the learning process of value function is detailed, covering the aspects of calculation of Q values, rollout decision rule and selection strategy of actions.

1) Calculation of Q values

In order to apply our RRHA to solve the DSP-UD, we reformulate the problem as an MDP. The complete disassembly process of a product can be decomposed into DL (i.e., disassembly depth) decision stages, corresponding to DL disassembly operations. At current decision stage dl ($1 \leq dl \leq DL$), the agent selects an available action t_j to enter the next stage $dl + 1$. Meanwhile, the current disassembly state s_{dl} transfers to the next state s_{dl+1} . Therefore, the MDP is represented as follow:

- $\mathcal{S} = \{s_1, \dots, s_{dl}, \dots, s_{DL}\}$ is a state set for all the stages during the disassembly process, where s_{dl} represents the state at the current decision stage dl .
- $\mathcal{A} = \{A_1, \dots, A_{dl}, \dots, A_{DL}\}$ is an action set for each stage, where A_{dl} indicates the set of the available transitions that can be executed at the current decision stage dl as an action. Additionally, symbol L_{dl} is defined to represent the cardinality of A_{dl} .
- $r(s, t_j)$ is a reward function that denotes the expected recovering profit after executing the action t_j at the current state s , which equals the expected difference of recovering revenue $E_r(s, t_j)$ and disassembly cost $E_c(s, t_j)$ shown in Equation (11).

$$r(s, t_j) = E_r(s, t_j) - E_c(s, t_j) \quad (11)$$

To formulate the $E_r(s, t_j)$ and $E_c(s, t_j)$, we introduce an incidence matrix G to represent the relation between the original places $p_i \in P_O$ and actions $t_j \in T_D$:

$$G = (O_{m \times n} - I_{n \times m}^T)^T \quad (12)$$

G_{ij} represents the value of the i^{th} row and j^{th} column of the G matrix, which is defined as:

$$G_{ij} = \begin{cases} 1, & \text{if place } p_i \text{ can be obtained by executing action } t_j \\ -1, & \text{if place } p_i \text{ can be separated by executing action } t_j \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

The value of $E_r(s, t_j)$ can be calculated according to Equation (14), integrating the recovering revenue that gain from the obtained places and loss from the separated places i.e. $\xi(p_i)$, which is shown in section 2.7).

$$E_r(s, t_j) = \sum_{i=0}^n G_{ij} \cdot \xi(p_i) \quad (14)$$

Subsequently, the value of $E_c(s, t_j)$ is computed according to Equation (15) considering the cost of disassembly operations i.e. $\tau(t_j)$, and EOL operations i.e. $\tau(e_i)$, which is shown in section 2.8).

$$E_c(s, t_j) = \tau(t_j) + \sum_{i=0}^n G_{ij} \cdot \tau(e_i) \quad (15)$$

As shown in Fig. 2, the Q value function is constructed from the historical disassembly information and serves for the execution of action selection module by estimating the cumulative rewards iteratively. In our RRHA, the value function based on Bellman equation is formulaically defined as Equation (16).

$$Q(s, t_j) = (1 - \alpha) \cdot Q(s, t_j) + \alpha \cdot [r(s, t_j) + \beta \cdot \max_{t'} Q(s', t')] \quad (16)$$

The parameter $\alpha \in (0, 1)$ represents the learning rate, denoting the learning efficiency for new disassembly information. $\beta \in (0, 1)$ is the discount factor and reflects the agent's preference between immediate rewards and long-term rewards. A higher discount factor emphasizes immediate recovering profit. The expression $\max_{t'} Q(s', t')$ represents the maximal Q value among all the state-action (state-transition) pairs that associated with the next state s' , which is transferred from the current state s through the selected action t_j . The value of state-action pair $Q(s, t_j)$ is initialized to 0, and eventually converges by iterative learning.

2) Rollout decision rule

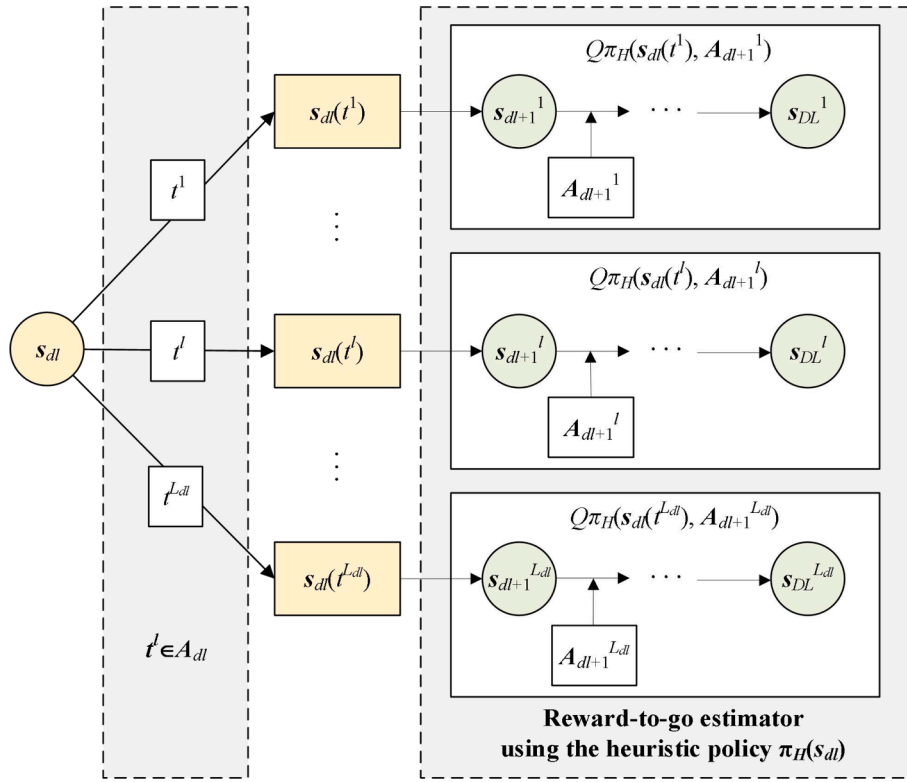


Fig. 3. The framework of rollout decision rule to estimate future profit.

Rollout algorithm is a kind of discrete near-optimal method. The rollout decision rule refers to a core strategy applied in the rollout algorithm, where an agent explores the long-term reward-to-go by simulating a sequence of possible actions from the current state to enhance the decision process. Before the mechanism of rollout decision rule to be demonstrated, it is necessary to introduce the concept of heuristic policy in advance, which is also called as a “base policy” in some literature.

The heuristic policy $\pi_H(s_{dl}) = \{\theta_H(s_{dl}), \theta_H(s_{dl+1}), \dots, \theta_H(s_{DL})\}$ is the initial policy based on heuristic H that serves for the action selection during the rollout process onward from the state s_{dl} to the terminal state s_{DL} . The definition of dl is number of current decision stage and DL is the number of final decision stage, which is mentioned in section 3.2.

In this paper, $\pi_H(s_{dl})$ is employed to estimate the future expected recovering profit for each available action t^l where $l = 1, 2, \dots, L_{dl}$ in the action set A_{dl} , which starts from the post-decision stage $s_{dl}(t^l)$. Therefore, the framework of rollout decision rule is shown in Fig. 3. It is worth noting that the expected value $E(s_{dl+1}^l, s_{DL}^l)$ from s_{dl+1}^l to s_{DL}^l can be converted to the $Q\pi_H(s_{dl}(t^l), A_{dl+1}^l)$ according to the action value function Equation (17), where A_{dl+1}^l represents the feasible action set at the next state s_{dl+1}^l . This expression represents the long-term cumulative expected reward of $s_{dl}(t^l)$ by previous iterative simulation. Thus, the process of rollout decision rule applied in the Q-learning framework can be formulated as the following Algorithm 1.

Algorithm 1 The action evaluation process of rollout decision rule

Input: current state s_{dl} , current feasible action set A_{dl}
Output: action value set V
Begin: Initialize $V \leftarrow \emptyset$;
1 for each action $t^l \in A_{dl}$ do
2 Transfer to next state s_{dl+1}^l from s_{dl} by t^l ;
3 Filter out the feasible action set A_{dl+1}^l at s_{dl+1}^l ;
4 Execute heuristic H to obtain $Q\pi_H(s_{dl}(t^l), A_{dl+1}^l)$;
5 $E(s_{dl+1}^l, s_{DL}^l) \leftarrow Q\pi_H(s_{dl}(t^l), A_{dl+1}^l)$;
6 $\varphi \leftarrow r(s_{dl}, t^l) + E(s_{dl+1}^l, s_{DL}^l)$;
7 Store φ into set V ;
8 end

We use the EPN in Fig. 1 to illustrate the process of the rollout decision rule. Suppose the current product is disassembled through transition t_2 (i.e., the current step $dl = 2$), the corresponding state s_{dl} is $\{0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0\}$. Then, the set of available action A_{dl} includes actions $\{t_4, t_5\}$. For these two available actions t_4 and t_5 , the rollout decision rule is applied to calculate the future reward-to-go. Taking t_4 as an example, the next state after executing t_4 is $s_{dl+1} = \{0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0\}$ and feasible action set A_{dl+1} includes $\{t_5, t_6, t_7\}$. Its future reward-to-go $= r(s_{dl}, t_4) + Q\pi_H(s_{dl}(t_4), A_{dl+1})$, where $r(s_{dl}, t_4)$ can be calculated according to Equation (12) and $Q\pi_H(s_{dl}(t_4), A_{dl+1})$ is determined based on Algorithm 1. Likewise, the future-reward-to-go of each feasible action at the current decision stage (i.e., t_4 and t_5) can be calculated.

3) Selection strategy of actions

To balance the global exploration and local exploitation effectively, we design the action selection strategy based on the Boltzmann probability distribution. This strategy is executed in the situations that 1) it serves as the heuristic H in rollout decision rule, 2) it is employed as the action selection at the current state s_{dl} based on the action value set V after executing Algorithm 1. The probability of selecting action $t^l \in A_{dl}$ at state s_{dl} is formulated as follows, where $\varphi(s_{dl}, t^l)$ represents the action value of taking action t^l at current state s_{dl} which is stored in V :

$$P(t^l | s_{dl}) = \frac{e^{\frac{\varphi(s_{dl}, t^l)}{\gamma \cdot T}}}{\sum_{t^l \in A_{dl}} e^{\frac{\varphi(s_{dl}, t^l)}{\gamma \cdot T}}} \quad (17)$$

The parameter γ is set to 100 to restrain the exponential growth. To control the learning process, the parameter T is calculated as Equation (18), where the $g_{current}$ is the current number of iterations and g_{max} is the maximal number of iterations. The value of T is large at first due to the small number of $g_{current}$. Subsequently, the algorithm gradually tends to local exploitation as the value of T decreases. Thus, the probability of selecting the state-action pair with the highest Q value will be

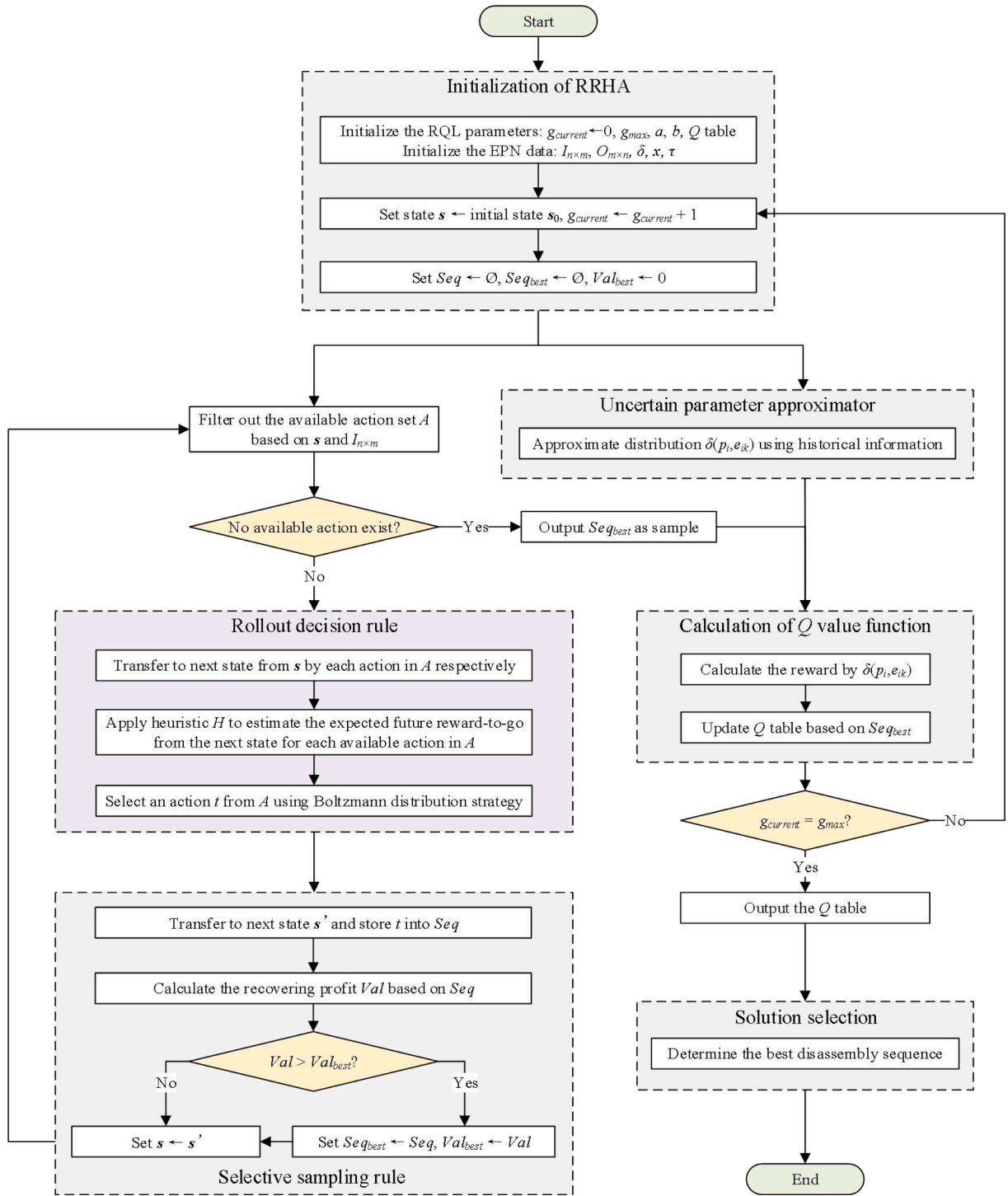


Fig. 4. The implementation procedure of RRHA.

significantly increased at the later phase of iteration.

$$T = 1 - \frac{0.99 \cdot g_{\text{current}}}{g_{\text{max}}} \quad (18)$$

3.3. Selective sampling rule of disassembly sequences

In our DSP-UD, the final disassembly state of the product is uncertain. Therefore, the algorithm requires the selective disassembly sequences as sample to enhance the estimate precision of cumulative rewards that considering uncertainty. Exactly, a sampled disassembly

sequence allows for the existence of any available action at final disassembly state during an iteration. Thus, we intercept the segments in the sampled disassembly sequence that can obtain the highest recovering profit to update the Q value during each iteration. Unlike the conventional approach that Q value is updated synchronously when each action is executed, the RRHA rewrites all the relevant state-action pairs after sampling the complete disassembly sequence. Eventually, the selective sampling rule of disassembly sequences is formulated as the following **Algorithm 2**.

(continued on next page)

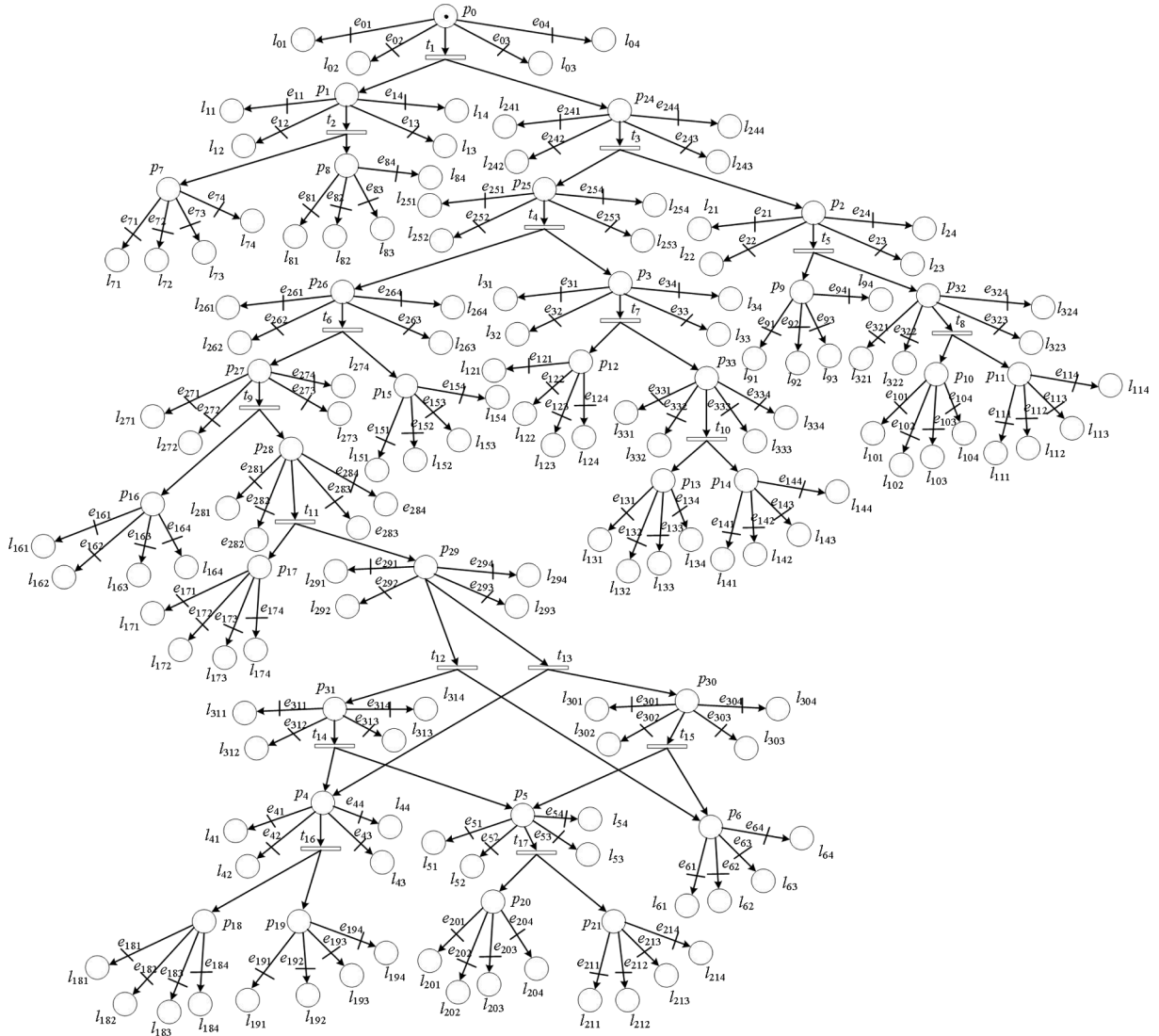


Fig. 5. The EPN of HLBP.

(continued)

Algorithm 2 The selective sampling rule of disassembly sequences**Algorithm 2** The selective sampling rule of disassembly sequences

Input: initial state s_0 , Q table
Output: updated Q table
Begin: Initialize current disassembly sequence $Seq \leftarrow \emptyset$,
 and best disassembly sequence $Seq_{best} \leftarrow \emptyset$,
 and maximum recovering profit $Val_{best} \leftarrow 0$;

- 1 Set current state $s_{dl} \leftarrow s_0$;
- 2 Filter out the feasible action set A_{dl} at s_{dl} ;
- 3 **while** 1
- 4 Select the action $t^* \in A_{dl}$ according to the action selection strategy
 and rollout decision rule, store t^* into Seq ;
- 5 Transfer to next state s_{dl+1}^* from s_{dl} by t^* ;
- 6 Calculate the current recovering profit Val according to Seq ;
- 7 **if** $Val > Val_{best}$ **do**
- 8 $Seq_{best} \leftarrow Seq$;
- 9 $Val_{best} \leftarrow Val$;
- 10 **end**
- 11 Filter out the feasible action set A_{dl+1}^* at next state s_{dl+1}^* ;
- 12 **if** $A_{dl+1}^* == \emptyset$ **do**
- 13 **break**
- 14 **else**
- 15 $s_{dl} \leftarrow s_{dl+1}^*$;
- 16 $A_{dl} \leftarrow A_{dl+1}^*$;
- 17 **continue**

(continued on next column)

(continued)

Algorithm 2 The selective sampling rule of disassembly sequences

- 18 **end**
- 19 **end**
- 20 Update Q table according to Seq_{best} and Val_{best}

3.4. Selection method of disassembly solution

After the learning process described from [section 3.1](#) to [section 3.3](#), the resultant Q table is convergent. A selection method is developed to obtain the best disassembly sequence from the Q table. The best solution is determined by executing the following **Algorithm 3**.

Algorithm 3 selection method for best disassembly sequence

Input: initial state s_0 , convergent Q table
Output: Seq_{best} , Val_{best}
Begin: Initialize current disassembly sequence $Seq \leftarrow \emptyset$,
 and best disassembly sequence $Seq_{best} \leftarrow \emptyset$,
 and maximum recovering profit $Val_{best} \leftarrow 0$;

- 1 Set current state $s_{dl} \leftarrow s_0$;
- 2 Filter out the feasible action set A_{dl} at s_{dl} ;
- 3 **while** 1
- 4 Select the action $t^* \in A_{dl}$ with maximum Q value, store t^* into Seq ;

(continued on next page)

Table 3
Bill of materials for HLBP.

Product level	Place	Composition
Li-ion battery pack	p_0	$\{1, 2, 3, 4, 5, 6 \times 2, 7, 8, 9 \times 4, 10 \times 16, 11 \times 32, 12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
Component level		
Upper housing	p_1	$\{1, 2\}$
Battery junction box	p_2	$\{3, 4, 5\}$
Battery Management Controller (BMC)	p_3	$\{6 \times 2, 7, 8\}$
Cooling system	p_4	$\{12 \times 4, 13 \times 2\}$
Lower housing	p_5	$\{14, 15\}$
Battery module	p_6	$\{16, 17\} \times 8$
Part level		
Upper housing shell	p_7	1
Upper insulator	p_8	2
Plug-in cable between BJB and CMCs-BMC	p_9	3
Battery Junction Box (BJB)	p_{10}	4
High Voltage cables and connectors	p_{11}	5
Top transverse cover	p_{12}	6×2
Plug-in cable CMCs-BMC	p_{13}	7
BMC	p_{14}	8
Module connector	p_{15}	9×4
Side module junction	p_{16}	10×16
Top and bottom fastener of module	p_{17}	11×32
Cooling plate	p_{18}	12×4
Cooling pipe	p_{19}	13×2
Lower insulation	p_{20}	14
Lower housing shell (aluminium)	p_{21}	15
Battery cell	p_{22}	16×8
Cell Management Controller (CMC)	p_{23}	17×8
Other auxiliary subassemblies		
	p_{24}	$\{3, 4, 5, 6 \times 2, 7, 8, 9 \times 4, 10 \times 16, 11 \times 32, 12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
	p_{25}	$\{6 \times 2, 7, 8, 9 \times 4, 10 \times 16, 11 \times 32, 12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
	p_{26}	$\{9 \times 4, 10 \times 16, 11 \times 32, 12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
	p_{27}	$\{10 \times 16, 11 \times 32, 12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
	p_{28}	$\{11 \times 32, 12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
	p_{29}	$\{12 \times 4, 13 \times 2, 14, 15, \{16, 17\} \times 8\}$
	p_{30}	$\{14, 15, \{16, 17\} \times 8\}$
	p_{31}	$\{12 \times 4, 13 \times 2, 14, 15\}$
	p_{32}	$\{4, 5\}$
	p_{33}	$\{7, 8\}$

(continued)

Algorithm 3 selection method for best disassembly sequence	
5	Transfer to next state s_{dl+1}^* from s_{dl} by t^* ;
6	Calculate the current recovering profit Val according to Seq ;
7	if $Val > Val_{best}$ do
8	$Seq_{best} \leftarrow Seq$;
9	$Val_{best} \leftarrow Val$;
10	End
11	Filter out the feasible action set A_{dl+1}^* at s_{dl+1}^* ;
12	if $A_{dl+1}^* == \emptyset$ do
13	Break
14	Else
15	$s_{dl} \leftarrow s_{dl+1}^*$;
16	$A_{dl} \leftarrow A_{dl+1}^*$;
17	Continue
18	End
19	End

3.5. Implementation procedure of RRHA

In the previous subsections, we formally describe the key modules of the proposed RRHA. Finally, the implementation procedure of RRHA is shown in Fig. 4.

4. Experimental results

To determine the disassembly performance of the proposed EPN model and RRHA, the hybrid Li-ion battery pack (hereinafter the HLBP) from Audi A3 Sportback e-tron is selected for our research in this section, which was used by Alfaro-Algaba and Ramirez [38]. Subsequently, we construct the EPN model of HLBP and present the experimental results of RRHA. All the programs are coded in MATLAB R2019a under Windows 11 operating system and run on a laptop computer with Intel (R) Core (TM) i7-9750 CPU@2.60 GHz with 16.00 GB RAM. We first validated the optimization effectiveness of the RRHA under various parameter settings. Additionally, we conducted two sets of comparative experiments to assess the performance of RRHA, including the comparisons with two different reinforcement learning algorithms and three typical heuristic algorithms.

4.1. HLBP

The HLBP consists of 17 distinct parts (excluding fasteners), differentiated using numbers 1 to 17 as label. The EPN based on HLBP is presented in Fig. 5, comprising 159 places (including 34 original places and 125 terminal places) and 145 transitions (including 17 disassembly operations and 128 EOL operations). Table 3 illustrates the Bill of Materials (BOM) for HLBP. The first and the second column of Table SI classifies the subassemblies into three levels with place numbers: product level (i.e., p_0), component level (i.e., p_1 to p_6), and part level (i.e., p_7 to p_{23}), together with the other auxiliary subassemblies (i.e., p_{24} to p_{33}) for facilitating the construction of EPN. The last column of Table SI displays the parts contained within each subassembly, where the integers preceding symbol “ $\times$ ” indicates the indices of parts and the integers following symbol “ $\times$ ” indicates the quantity of the corresponding parts. If there is no symbol “ $\times$ ”, it means the part is single. The same subassemblies are incorporated into one place to simplify the EPN model. For instance, all the battery modules in HLBP are integrated into p_6 .

Some other crucial disassembly information for HLBP is obtained from literature [38], including the data of recovering revenue ξ for places and the cost τ for transitions, which refer to section 2.7) and section 2.8).

4.2. Experimental results of RRHA

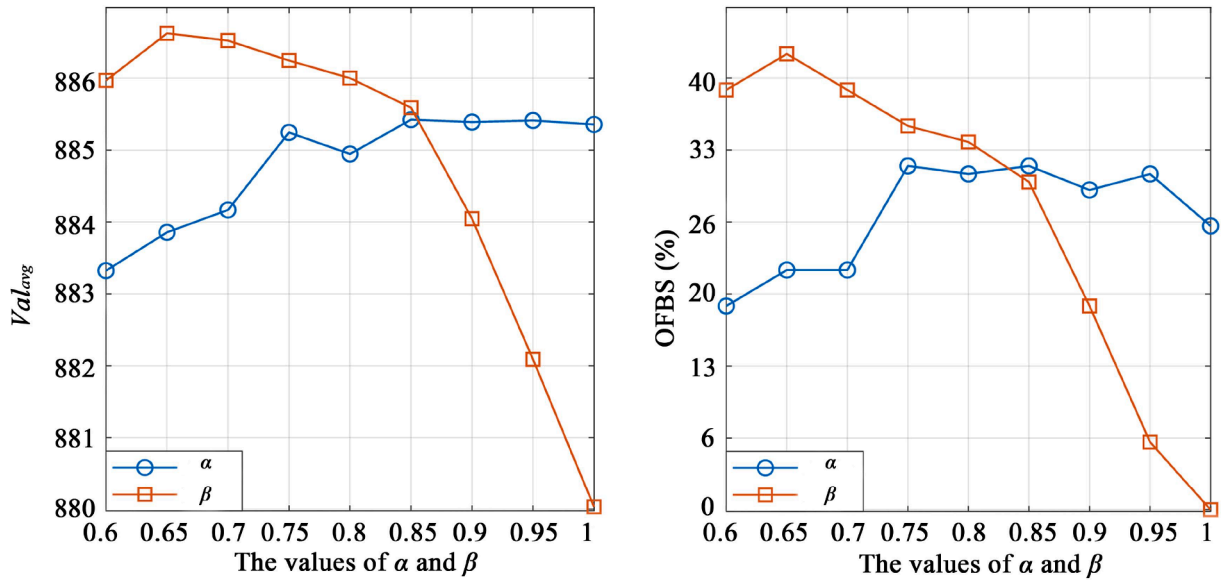
To determine the ideal parameter combination, the tuning parameter experiment refers to [39,40] and is conducted by setting the following parameters independently: $g_{max} = \{1000, 2000, 3000\}$, $\alpha = \{0.25, 0.50, 0.75\}$, and $\beta = \{0.25, 0.50, 0.75, 1.00\}$ so that a total of 36 combinations (i.e., $3 \times 4 \times 3$) are tested for the initial tuning parameter process. For each parameter combination, the RRHA is implemented on the EPN of HLBP for 30 repetitions to evaluate the disassembly performance. The average recovering profit Val_{avg} are presented in Table 4, with the best recovering profit Val_{max} of 892.3 corresponding to the operation sequence $t_1 \rightarrow t_3 \rightarrow t_4 \rightarrow t_6 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12} \rightarrow t_{14} \rightarrow t_{16} \rightarrow t_{17}$.

From Table 4, it is obvious that the individual effects of learning rate α and discount factor β on recovering profit are not significant. However, we can find that higher profit is generally achieved when the values of α and β are equal. This situation exists even g_{max} takes 3000. The maximum average profit is 888.3, which appears at the parameter

Table 4

The computational results of RRHA at the initial tuning parameters.

g_{max}	β	$\alpha = 0.25$			$\alpha = 0.50$			$\alpha = 0.75$		
		Val_{avg}	CPU (s)	OFBS (%)	Val_{avg}	CPU (s)	OFBS (%)	Val_{avg}	CPU (s)	OFBS (%)
1000	0.25	882	9	3.3	876	9.1	0	874.1	9.1	0
	0.5	879	9	0	881.5	9.1	13.3	879.8	9.1	6.7
	0.75	878.6	8.9	3.3	879.5	9	0	883	9	13.3
	1	875.9	8.8	0	879	8.8	0	879.1	8.8	0
2000	0.25	884.3	18.5	23.3	877.7	18.6	6.7	876.2	18.4	0
	0.5	882.9	18.5	13.3	883.1	18.5	20	880.9	18.6	6.7
	0.75	880.9	18.4	3.3	883.1	18.4	10	885.3	18.5	23.3
	1	877.3	18.3	0	879	18.4	0	879.3	18.4	0
3000	0.25	885.1	27.8	36.7	882.8	28.1	23.3	876.5	28	0
	0.5	881.5	27.8	10	886	28	43.3	883.3	28.1	16.7
	0.75	882.3	27.7	10	884	27.9	16.7	888.3	27.8	53.3
	1	876.7	28	0	879.1	28	0	879	27.9	0

**Fig. 6.** The fine-tuning results of α and β on Val_{avg} (left) and OFBS (right).

settings of $\{g_{max} = 3000, \alpha = 0.75, \beta = 0.75\}$. The column “OFBS (%)” in Table 4 presents the Occurrence Frequency of Best Solution (OFBS) obtained by each parameter combination in 30 repetitions, reflecting the global search capability of the RRHA. It is worth noting that higher frequencies still occur when the α is to β under various iteration of g_{max} .

Similarly, the settings of $\{g_{max} = 3000, \alpha = 0.75, \beta = 0.75\}$ yields the highest frequency of best solution i.e. 53.3 %.

Further, we employ a grid search method to fine-tune the parameters based on the ranges of $\alpha \in [0.6, 1]$ and $\beta \in [0.6, 1]$. α and β are fine-tuned by a step size of 0.05, and a total of 81 parameter combinations are thus

Table 5

The statistical results of the best disassembly sequence using each parameter combination.

No.	The best disassembly sequence found by 81 parameter combinations	Parameter combinations	Frequency	Recovering profit
1	$t_1 \rightarrow t_3 \rightarrow t_4 \rightarrow t_6 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12} \rightarrow t_{14} \rightarrow t_{16} \rightarrow t_{17}$	1-8, 10-17, 19-25, 28-35, 37-44, 46-52, 55-62, 64-70, 73-80	69	892.3
2	$t_1 \rightarrow t_3 \rightarrow t_4 \rightarrow t_5 \rightarrow t_6 \rightarrow t_8 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12} \rightarrow t_{14} \rightarrow t_{16} \rightarrow t_{17}$	54, 71	2	888.8
3	$t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow t_6 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12} \rightarrow t_{14} \rightarrow t_{16} \rightarrow t_{17}$	26, 36, 53, 63, 72, 81	6	883.0
4	$t_1 \rightarrow t_3 \rightarrow t_4 \rightarrow t_6 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12}$	18, 27, 45	3	880.9
5	$t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow t_5 \rightarrow t_6 \rightarrow t_8 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12} \rightarrow t_{14} \rightarrow t_{16} \rightarrow t_{17}$	9	1	879.5

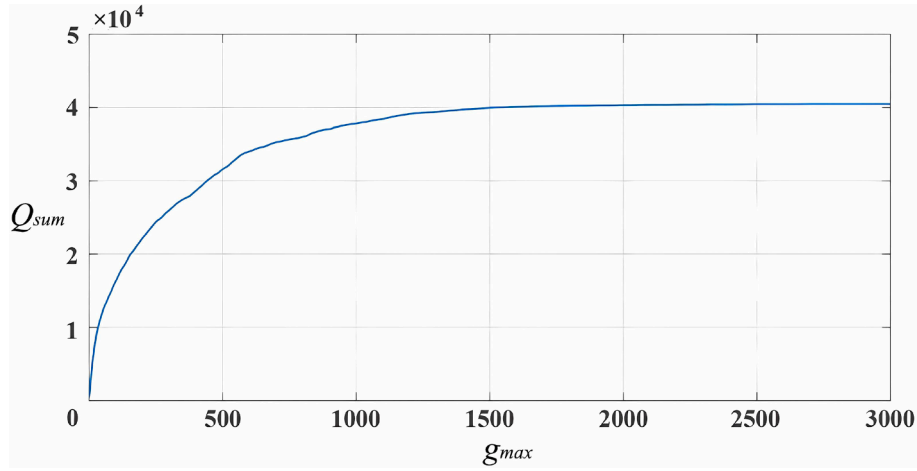


Fig. 7. The evolution of Q_{sum} under the best parameter combination.

tested. By running RRHA 30 times, the computational results of RRHA on 81 parameter combinations are shown in Table A1 of Appendix. It can be observed that the parameter settings of $\{g_{max} = 3000, \alpha = 0.75, \beta = 0.75\}$ remains the best combination across all evaluation metrics. Subsequently, we employ this combination in the following comparative experiments in section 4.3 with to balance the solution quality and computational cost. Fig. 6 demonstrates the impact of fine-tuning learning rate α and discount factor β on the performance of RRHA algorithm after 3000 iterations. as the value of α increases, the performance of RRHA improves. However, as the discount factor β increases especially $\beta \in [0.85, 1]$, the algorithm performance shows a significant decreasing trend.

To show the robustness of RRHA under different parameter settings, the best disassembly sequence for each parameter combination in Table A1 is selected after running RRHA 30 times, whose statistical results are presented in Table 5. In the table, the second column lists five different best disassembly sequences found by 81 parameter combinations. The second and third columns describe the corresponding parameter combinations (as the same with Table A1) and their frequencies, respectively. It can be seen that the first best disassembly sequence, i.e., $t_1 \rightarrow t_3 \rightarrow t_4 \rightarrow t_6 \rightarrow t_9 \rightarrow t_{11} \rightarrow t_{12} \rightarrow t_{14} \rightarrow t_{16} \rightarrow t_{17}$, are identified 69 out of 81 parameter settings. This effectively demonstrates that the proposed method is little influenced by tuning α and β between $[0.6, 1]$. By comparing with these best disassembly sequences, we can find that they are highly similar with each other except the fourth disassembly sequence, which again shows the good robustness of RRHA when taking α and β in the interval $[0.6, 1]$.

An indicator Q_{sum} is introduced to verify the convergence of RRHA, which equals to the sum of values for all state-action pairs in the Q table. Under the conditions of $\{g_{max} = 3000, \alpha = 0.75, \beta = 0.75\}$, the average Q_{sum} during iterations is illustrated in Fig. 7. Finally, the evolution of Q_{sum} tends to be stable after the 1500th iteration and nearly converges to 40,500 after the 2200th iteration.

4.3. Comparison results

To further validate the performance of the proposed algorithm, we compare RRHA with two other different reinforcement learning methods and three typical heuristic methods. The comparison results are given below.

1) Comparisons with other RL methods

To show the superiority of the rollout decision rule used in RRHA, two other existing reinforcement learning algorithm, i.e., self-adaptive learning approach (SALA) [40] and deep Q-network (DQN) [30] are

Table 6

The computational results using different RL methods.

Indicator	RRHA	SALA	DQN
Val_{avg}	888.3	866.8	887.1
OFBS (%)	53.3	20	36.7
CPU (s)	27.8	24.7	21.9

adopted to compare with the proposed algorithm. The comparative results of 3000 iterations are presented in Table 6. Comparing to the results of applying RRHA, the other two methods demonstrate inferior ability across the average profit and the OFBS. However, the RRHA consumes slightly more average CPU time than SALA and DQN under all the conditions showed in Fig. 8 due to the rollout simulation process.

For the global search capability of the three algorithms, the SALA and DQN struggle to achieve best disassembly solutions. It can be observed that the RRHA exhibits significantly stronger global search capability than other two methods. In addition, Fig. 9 shows that rollout decision rule obtains the best average OFBS significantly among the above methods. In summary, the RRHA can achieve the highest average profit, while its global search ability surpasses the other two other methods applied in previous studies.

2) Comparisons with other (meta)heuristic methods

To further demonstrate the performance of RRHA, three existing heuristic methods are applied to compare with it. The first one is genetic algorithm (GA). It is one of the most commonly used method for solving the DSP. We modify the GA algorithm proposed in [22] as single objective. The second method is Scatter Search (SS) used in [41]. The last one is Teaching-Learning-Based Optimization (TLBO) [42]. Due to the difficulty of heuristic algorithms in addressing uncertainties during iterations, we directly utilize the probabilities $\delta(p_i, e_{ik})$ derived from the sample training of RRHA for profit calculation, thereby transforming it into a deterministic DSP. Each heuristic method is repeated 30 times after tuning the parameter settings, and their computational results against RRHA are shown in Table 7.

It must be stressed that an approximately similar CPU time are used for each algorithm in Table 7 to evaluate the solution quality. From the table, we can observe that RRHA is superior to other heuristic methods in terms of both Val_{avg} and OFBS. In particular, OFBS of our proposed approach reaches 53.3 % that is much higher than GA (16.7 %), SS (20.0 %), and TLBO (13.3 %). This indicates that RRHA is significantly easier to search for the best solution within the almost similar computational time than other three heuristic methods.

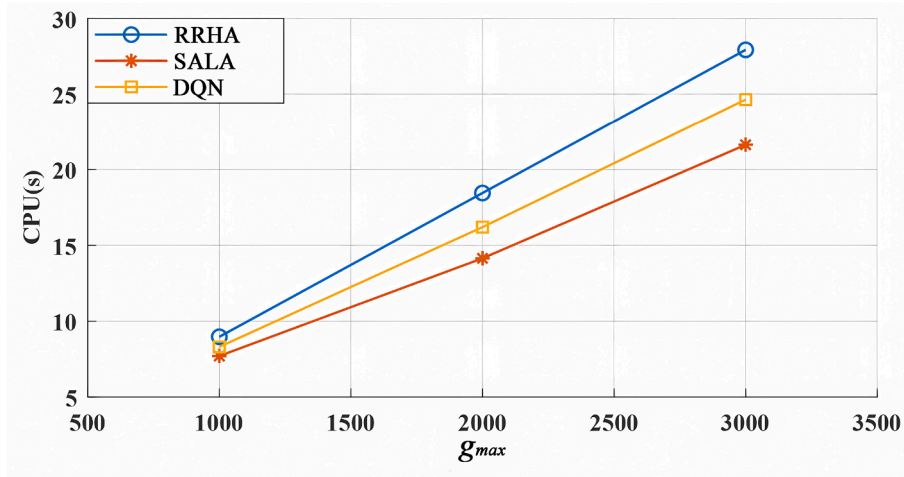


Fig. 8. The CPU time of different RL methods with the variation of g_{max} .

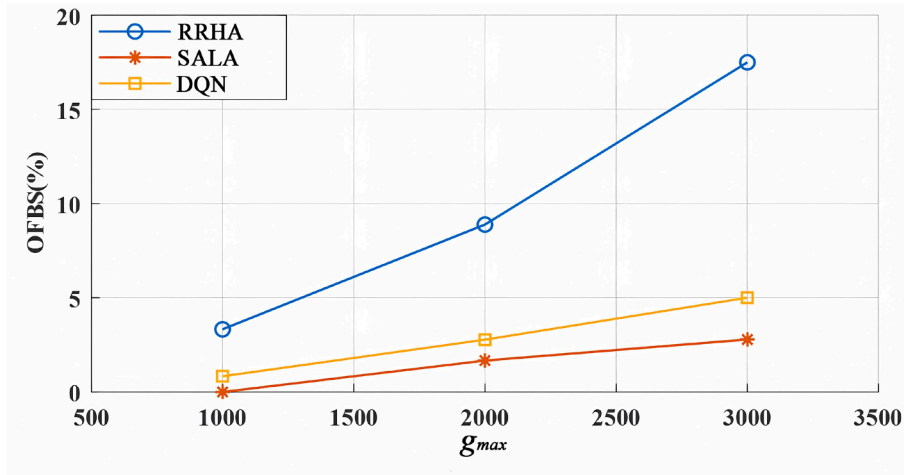


Fig. 9. OFBS of different RL methods with the variation of g_{max} .

Table 7

The computational results of RRHA, GA, SS, and TLBO.

Indicator	RRHA	GA	SS	TLBO
Val_{avg}	888.3	888.0	887.6	886.9
OFBS (%)	53.3	16.7	20.0	13.3
CPU (s)	27.8	26.9	27.5	27.3

Table 8

The computational results of the proposed RRHA with different products.

Product	Number of places	Number of transitions	OFBS (%)	CPU (s)	MDD
HLBP	34	17	53.3	27.8	15
radio	29	30	60.0	20.7	9
transmission	34	39	46.7	32.3	19

4.4. Discussion and limitation

In this segment, additional two product instances with different complexities and sizes, along with HLBP used in the previous subsections, are utilized to verify the applicability and versatility of the proposed method. The first product is a radio with 29 subassemblies and 30 operations and another product is a transmission with 34

subassemblies and 39 operations [43]. Also, an indicator, i.e., maximum disassembly depth (MDD) is used to evaluate the method performance, which represents the number of disassembly operations required for the complete disassembly. Table 8 provides the computational results of RRHA for three product instances.

From Table 8, we can see that the OFBS values of three product instances are more than 45 %, which indicates that RRHA can easily find the best disassembly sequence across various disassembly environments. However, the OFBS values are trended to decreased as the MDD values grow. This demonstrates that the versatility and robustness of the proposed method are influenced by MDD of the product instance. Such a similar phenomenon occurs in terms of the computational cost, i.e., the computational time also increases with the increment of MDD. Fortunately, the computational cost used by RRHA is acceptable. Even for the transmission with the largest MDD, its CPU time is 32.3 s.

By further analyzing the algorithm procedure of RRHA, it can be observed that each iteration of the learning process would be terminated only when a complete disassembly is done for the product and the best partial disassembly sequence (i.e., a subsequence of the complete disassembly sequence) is afterward determined based on the complete disassembly sequence, as mentioned in Section 3.3. For this reason, the computational efficiency of the proposed method is limited to MDD of the product to a certain extent.

5. Conclusions

This work develops a hybrid algorithm, i.e., RRHA to search for the high-quality disassembly solutions for a disassembly sequence planning with uncertain depreciation and diversified recovering strategies, in which an extended Petri net is established to model our problem and a rollout decision rule is integrated into the reinforcement learning framework to select the actions. Ultimately, the performance of RRHA is examined by a hybrid Li-ion battery pack from Audi A3 Sportback e-tron. By comparing with other two reinforcement learning methods and three typical heuristic methods, RRHA achieves the highest average recovering profit (i.e., $Val_{avg} = 888.3$) and exhibits the highest frequency for identifying the best solution (i.e., OFBS = 53.3 %), which effectively demonstrates the superiority of RRHA in solving our DSP problem.

Additional two products with different complexities and sizes are used to further test the proposed method. The experimental results show that the computational performance of RRHA is influenced when dealing with the product with a large disassembly depth. Therefore, we will study how to solve this limitation and improve the proposed method in the future. Also, the rollout decision rule is quite suitable to address a dynamic DSP problem by integrating reinforcement learning, which is worthy to be further studied in the future work.

CRedit authorship contribution statement

Yaping Ren: Writing – original draft, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Formal

analysis, Data curation, Conceptualization. **Zhehao Xu:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation. **Yanzi Zhang:** Writing – original draft, Project administration, Methodology, Funding acquisition, Conceptualization. **Jiayi Liu:** Writing – review & editing, Writing – original draft, Resources. **Leilei Meng:** Writing – original draft, Data curation. **Wenwen Lin:** Writing – original draft, Methodology, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Funds for National Natural Science Foundation of China (Grant Nos. 52205526, 52205529, 52205538, 72371028, and 72101133), Foundation of State Key Laboratory of Public Big Data (Grant No. QJJ[2022]418), and the Fundamental Research Funds for the Central Universities (Grant No. 21623219). We also acknowledge the support from the Fundamental Research Funds for the Central Universities, Beijing Institute of Technology. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Natural Science Foundation of China.

Appendix

Table A1

The computational results of RRHA on 81 fine-tuning parameter combinations.

No.	g_{max}		1000		2000		3000	
	α	β	Val_{avg}	OFBS (%)	Val_{avg}	OFBS (%)	Val_{avg}	OFBS (%)
1	0.6	0.6	880.5	10.0	882.1	23.3	885.8	43.3
2		0.65	880.1	3.3	882.1	16.7	885.6	33.3
3		0.7	879.5	0.0	881.2	10.0	884.6	20.0
4		0.75	879.1	0.0	879.9	0.0	882.4	6.7
5		0.8	879.9	3.3	881.2	6.7	883.7	20.0
6		0.85	880.2	3.3	882.7	13.3	884.5	26.7
7		0.9	880.1	0.0	881.5	3.3	883.0	13.3
8		0.95	878.3	0.0	879.8	0.0	880.8	6.7
9		1	876.5	0.0	879.2	0.0	879.5	0.0
10	0.65	0.6	880.9	6.7	883.9	13.3	885.5	36.7
11		0.65	881.6	13.3	884.3	20.0	886.7	46.7
12		0.7	880.9	6.7	883.4	16.7	885.5	40.0
13		0.75	879.7	3.3	882.7	10.0	884.4	20.0
14		0.8	880.1	3.3	881.2	3.3	883.4	13.3
15		0.85	880.0	0.0	881.6	6.7	884.2	23.3
16		0.9	879.9	0.0	881.5	0.0	883.5	13.3
17		0.95	879.2	0.0	880.4	0.0	881.9	6.7
18		1	876.8	0.0	878.6	0.0	879.6	0.0
19	0.7	0.6	879.5	6.7	882.8	16.7	886.0	33.3
20		0.65	881.6	6.7	883.3	20.0	885.6	33.3
21		0.7	881.9	10.0	884.9	23.3	887.0	43.3
22		0.75	881.2	3.3	882.7	10.0	885.2	33.3
23		0.8	880.2	0.0	883.2	10.0	885.4	26.7
24		0.85	880.5	3.3	883.6	6.7	885.3	23.3
25		0.9	880.4	0.0	881.5	0.0	882.7	6.7
26		0.95	879.8	0.0	880.1	0.0	880.7	0.0
27		1	877.9	0.0	879.6	0.0	879.6	0.0
28	0.75	0.6	881.2	6.7	883.5	20.0	886.9	43.3

(continued on next page)

Table A1 (continued)

No.	g_{max}		1000		2000		3000	
	α	β	Val_{avg}	OFBS (%)	Val_{avg}	OFBS (%)	Val_{avg}	OFBS (%)
29		0.65	881.6	6.7	884.3	16.7	887.2	46.7
30		0.7	881.9	13.3	884.9	23.3	887.6	46.7
31		0.75	881.2	13.3	885.5	26.7	888.7	56.7
32		0.8	880.2	3.3	882.2	13.3	885.8	33.3
33		0.85	880.5	3.3	882.1	10.0	884.0	20.0
34		0.9	880.4	6.7	881.3	16.7	884.6	26.7
35		0.95	879.8	3.3	880.8	6.7	882.5	13.3
36		1	878.9	0.0	879.5	0.0	879.9	0.0
37	0.8	0.6	879.7	10.0	881.8	23.3	884.9	33.3
38		0.65	880.1	10.0	881.6	20.0	885.2	30.0
39		0.7	880.3	6.7	882.5	20.0	885.7	33.3
40		0.75	882.5	10.0	885.1	20.0	887.9	53.3
41		0.8	881.3	13.3	884.2	26.7	887.5	50.0
42		0.85	880.4	6.7	883.1	16.7	887.0	40.0
43		0.9	879.1	3.3	882.7	10.0	884.8	33.3
44		0.95	878.6	0.0	880.5	0.0	881.7	6.7
45		1	877.2	0.0	879.6	0.0	879.8	0.0
46	0.85	0.6	879.2	6.7	882.8	20.0	886.0	46.7
47		0.65	879.9	10.0	883.1	20.0	886.5	43.3
48		0.7	882.3	10.0	885.1	13.3	887.7	43.3
49		0.75	881.5	13.3	884.4	23.3	886.3	33.3
50		0.8	882.8	10.0	884.2	16.7	886.9	46.7
51		0.85	882.4	13.3	885.6	26.7	888.1	53.3
52		0.9	880.1	3.3	883.5	6.7	884.7	20.0
53		0.95	879.4	0.0	880.2	0.0	882.4	0.0
54		1	877.5	0.0	879.9	0.0	880.2	0.0
55	0.9	0.6	878.6	3.3	881.6	13.3	884.9	20.0
56		0.65	881.5	10.0	884.2	23.3	887.8	53.3
57		0.7	881.2	10.0	883.6	23.3	886.3	40.0
58		0.75	881.8	6.7	883.5	16.7	886.2	26.7
59		0.8	882.4	13.3	884.8	20.0	887.1	40.0
60		0.85	881.9	6.7	884.1	16.7	885.1	33.3
61		0.9	882.2	13.3	884.9	23.3	886.6	40.0
62		0.95	879.6	3.3	882.3	3.3	884.1	13.3
63		1	879.2	0.0	879.8	0.0	880.4	0.0
64	0.95	0.6	880.2	10.0	883.8	20.0	886.6	46.7
65		0.65	881.1	6.7	884.5	16.7	887.5	53.3
66		0.7	881.7	10.0	883.4	20.0	887.4	46.7
67		0.75	882.3	3.3	884.1	13.3	887.6	46.7
68		0.8	882.5	6.7	883.7	13.3	887.2	46.7
69		0.85	880.4	6.7	882.9	20.0	886.1	33.3
70		0.9	880.6	3.3	882.4	3.3	883.2	6.7
71		0.95	879.0	0.0	881.1	0.0	882.0	0.0
72		1	878.5	0.0	879.7	0.0	881.1	0.0
73	1	0.6	881.6	10.0	884.6	23.3	887.1	46.7
74		0.65	882.4	13.3	885.7	23.3	887.5	40.0
75		0.7	881.5	6.7	883.9	13.3	886.9	36.7
76		0.75	882.8	13.3	884.3	20.0	887.5	43.3
77		0.8	881.2	6.7	884.1	16.7	887.0	30.0
78		0.85	881.1	6.7	883.7	13.3	886.0	20.0
79		0.9	879.6	0.0	882.1	3.3	883.3	10.0
80		0.95	879.8	3.3	881.0	3.3	882.7	10.0
81		1	879.1	0.0	879.3	0.0	880.2	0.0

Data availability

The data that has been used is confidential.

References

- [1] N.T. Ching, M. Ghobakhloo, M. Iranmanesh, P. Maroufkhani, S. Asadi, Industry 4.0 applications for sustainable manufacturing: a systematic literature review and a roadmap to sustainable development, *J. Clean. Prod.* 334 (2022) 130133.
- [2] Y. Ren, C. Zhang, F. Zhao, M.J. Triebe, L. Meng, An MCDM-based multiobjective general variable neighborhood search approach for disassembly line balancing problem, *IEEE Trans. Syst., Man, Cybernetics: Syst.* 50 (10) (2018) 3770–3783.
- [3] S.K. Singh, A. Chauhan, B. Sarkar, Supply chain management of E-waste for end-of-life electronic products with reverse logistics, *Mathematics* 11 (1) (2023) 124.
- [4] International Telecommunication Union. “4 key takeaways from the new Global E-waste Monitor 2020.” ITU News. <https://news.itu.int/4-key-takeaways-from-the-new-global-e-waste-monitor-2020/> (accessed).
- [5] European Commission Environment. “Waste from Electrical and Electronic Equipment (WEEE).” https://environment.ec.europa.eu/topics/waste-and-recycling/waste-electrical-and-electronic-equipment-weee_en (accessed).

- [6] OECD. "Extended Producer Responsibility." <https://www.oecd.org/environment/extended-producer-responsibility.htm> (accessed).
- [7] Y. Ren, et al., Disassembly line balancing problem using interdependent weights-based multi-criteria decision making and 2-Optimal algorithm, *J. Clean. Prod.* 174 (2018) 1475–1486.
- [8] W. Liang, et al., Improved optimisation method considering full solution space for disassembly line balancing problem in remanufacturing system, *Adv. Eng. Informatics* 58 (2023) 102237, <https://doi.org/10.1016/j.aei.2023.102237>.
- [9] A. Shreyas Madhav, R. Rajaraman, S. Harini, C.C. Kilroor, Application of artificial intelligence to enhance collection of E-waste: a potential solution for household WEEE collection and segregation in India, *Waste Manag. Res.* 40 (7) (2022) 1047–1053.
- [10] Y. Ren, et al., A multiobjective disassembly planning for value recovery and energy conservation from end-of-life products, *IEEE Trans. Autom. Sci. Eng.* 18 (2) (2020) 791–803.
- [11] Y. Ren, D. Yu, C. Zhang, G. Tian, L. Meng, X. Zhou, An improved gravitational search algorithm for profit-oriented partial disassembly line balancing problem, *Int. J. Prod. Res.* 55 (24) (2017) 7302–7316.
- [12] L. Guo, Z. Zhang, X. Zhang, Human-robot collaborative partial destruction disassembly sequence planning method for end-of-life product driven by multi-failures, *Adv. Eng. Informatics* 55 (2023) 101821, <https://doi.org/10.1016/j.aei.2022.101821>.
- [13] H.-E. Tseng, C.-C. Chang, S.-C. Lee, Y.-M. Huang, Hybrid bidirectional ant colony optimization (hybrid BACO): an algorithm for disassembly sequence planning, *Eng. Appl. Artif. Intel.* 83 (2019) 45–56.
- [14] H.-E. Tseng, C.-C. Chang, T.-W. Chung, Applying improved particle swarm optimization to asynchronous parallel disassembly planning, *IEEE Access* 10 (2022) 80555–80564.
- [15] G. Tian, Y. Ren, Y. Feng, M. Zhou, H. Zhang, J. Tan, Modeling and planning for dual-objective selective disassembly using AND/OR graph and discrete artificial bee colony, *IEEE Trans. Ind. Inf.* 15 (4) (2018) 2456–2468.
- [16] J.C. Chen, Y.-Y. Chen, T.-L. Chen, Y.-C. Yang, An adaptive genetic algorithm-based and AND/OR graph approach for the disassembly line balancing problem, *Eng. Optim.* 54 (9) (2022) 1583–1599.
- [17] Y. Ren, et al., An efficient metaheuristics for a sequence-dependent disassembly planning, *J. Clean. Prod.* 245 (2020) 118644.
- [18] Y. Fu, M. Zhou, X. Guo, L. Qi, K. Sedraoui, Multiverse optimization algorithm for stochastic biobjective disassembly sequence planning subject to operation failures, *IEEE Trans. Syst., Man, Cybernetics: Syst.* 52 (2) (2021) 1041–1051.
- [19] T. Suzuki, T. Zanma, A. Inaba, S. Okuma, Learning control of disassembly Petri net-an approach with discrete event system theory, in: *Proceedings of IEEE International Conference on Robotics and Automation, IEEE*, 1996, pp. 184–191.
- [20] X. Guo, M. Zhou, S. Liu, L. Qi, Multiresource-constrained selective disassembly with maximal profit and minimal energy consumption, *IEEE Trans. Autom. Sci. Eng.* 18 (2) (2020) 804–816.
- [21] J. Mao, D. Hong, Z. Chen, M. Changhai, L. Weiwen, J. Wang, Disassembly sequence planning of waste auto parts, *J. Air Waste Manag. Assoc.* 71 (5) (2021) 607–619.
- [22] X. Guo, M. Zhou, S. Liu, L. Qi, Multiresource-constrained selective disassembly with maximal profit and minimal energy consumption, *IEEE Trans. Automation Sci. Eng.* 18 (2) (2021) 804–816, <https://doi.org/10.1109/tase.2020.2992220>.
- [23] J. Xiao, N. Anwer, W. Li, B. Eynard, C. Zheng, Dynamic Bayesian network-based disassembly sequencing optimization for electric vehicle battery, *CIRP J. Manuf. Sci. Technol.* 38 (2022) 824–835.
- [24] A.J. Lambert, Disassembly sequencing: a survey, *Int. J. Prod. Res.* 41 (16) (2003) 3721–3759.
- [25] Y. Feng, et al., Target disassembly sequencing and scheme evaluation for CNC machine tools using improved multiobjective ant colony algorithm and fuzzy integral, *IEEE Trans. Syst., Man, Cybernetics: Syst.* 49 (12) (2018) 2438–2451.
- [26] G. Tian, M. Zhou, P. Li, Disassembly sequence planning considering fuzzy component quality and varying operational cost, *IEEE Trans. Autom. Sci. Eng.* 15 (2) (2017) 748–760.
- [27] Y. Gao, S. Lou, H. Zheng, J. Tan, A data-driven method of selective disassembly planning at end-of-life under uncertainty, *J. Intel. Manuf.* 34 (2) (2023) 565–585, <https://doi.org/10.1007/s10845-021-01812-0>.
- [28] X. Zhang, et al., Selective disassembly sequence planning under uncertainty using trapezoidal fuzzy numbers: a novel hybrid metaheuristic algorithm, *Eng. Appl. Artif. Intel.* 128 (2024) 107459, <https://doi.org/10.1016/j.engappai.2023.107459>.
- [29] Y. Ren, X. Lu, H. Guo, Z. Xie, H. Zhang, C. Zhang, A review of combinatorial optimization problems in reverse logistics and remanufacturing for end-of-life products, *Mathematics* 11 (2) (2023) 298.
- [30] X. Zhao, C. Li, Y. Tang, J. Cui, Reinforcement learning-based selective disassembly sequence planning for the end-of-life products with structure uncertainty, *IEEE Rob. Autom. Lett.* 6 (4) (2021) 7807–7814.
- [31] H. Mao, Z. Liu, C. Qiu, Adaptive disassembly sequence planning for VR maintenance training via deep reinforcement learning, *Int. J. Adv. Manuf. Technol.* (2021) 1–10.
- [32] A. Amirmia, S. Keivanpour, A context-aware real-time human-robot collaborating reinforcement learning-based disassembly planning model under uncertainty, *Int. J. Production Res. Article; Early Access* (2023), <https://doi.org/10.1080/00207543.2023.2252526>.
- [33] J. Xiao, J. Gao, N. Anwer, B. Eynard, Multi-agent reinforcement learning method for disassembly sequential task optimization based on human-robot collaborative disassembly in electric vehicle battery recycling, *J. Manuf. Sci. Eng.* 145 (12) (2023) 121001, <https://doi.org/10.1115/1.4062235>.
- [34] M. Neves, M. Vieira, P. Neto, A study on a Q-Learning algorithm application to a manufacturing assembly problem, *J. Manuf. Syst., Article 59* (2021) 426–440, <https://doi.org/10.1016/j.jmsy.2021.02.014>.
- [35] W.K. Anuar, L.S. Lee, H.-V. Seow, S. Pickl, A multi-depot vehicle routing problem with stochastic road capacity and reduced two-stage stochastic integer linear programming models for rollout algorithm, *Mathematics* 9 (13) (2021) 1572.
- [36] J.C. Goodson, B.W. Thomas, J.W. Ohlmann, A rollout algorithm framework for heuristic solutions to finite-horizon stochastic dynamic programs, *Eur. J. Operational Res., Article 258* (1) (2017) 216–229, <https://doi.org/10.1016/j.ejor.2016.09.040>.
- [37] M. Zhou, Petri nets in flexible and agile automation, *Springer Science & Business Media*, 2012.
- [38] M. Alfaro-Algaba, F.J. Ramirez, Techno-economic and environmental disassembly planning of lithium-ion electric vehicle battery packs for remanufacturing, *Resour. Conserv. Recycl.* 154 (2020) 104461.
- [39] Y. Ren, K. Gao, Y. Fu, H. Sang, D. Li, Z. Luo, A novel Q-learning based variable neighborhood iterative search algorithm for solving disassembly line scheduling problems, *Swarm Evolutionary Comput.* 80 (2023) 101338, <https://doi.org/10.1016/j.swevo.2023.101338>.
- [40] Y. Ren, H. Guo, Y. Li, J. Li, L. Meng, A self-adaptive learning approach for uncertain disassembly planning based on extended petri net, *IEEE Trans. Ind. Inf.* 19 (12) (2023) 11889–11897.
- [41] X. Guo, S. Liu, M. Zhou, G. Tian, Dual-objective program and scatter search for the optimization of disassembly sequences subject to multiresource constraints, *IEEE Trans. Automation Sci. Eng., Article 15* (3) (2018) 1091–1103, <https://doi.org/10.1109/tase.2017.2731981>.
- [42] K. Xia, L. Gao, W. Li, K.-M. Chao, Disassembly sequence planning using a simplified teaching-learning-based optimization algorithm Sustain, *Manuf. Remanuf. Manag.: Process Plan., Optimization Applic.* 319 (2019) 343.
- [43] Y. Ren, et al., An efficient metaheuristics for a sequence-dependent disassembly planning, *J. Clean. Production* 245 (2020), <https://doi.org/10.1016/j.jclepro.2019.118644>.